Registers and Memory

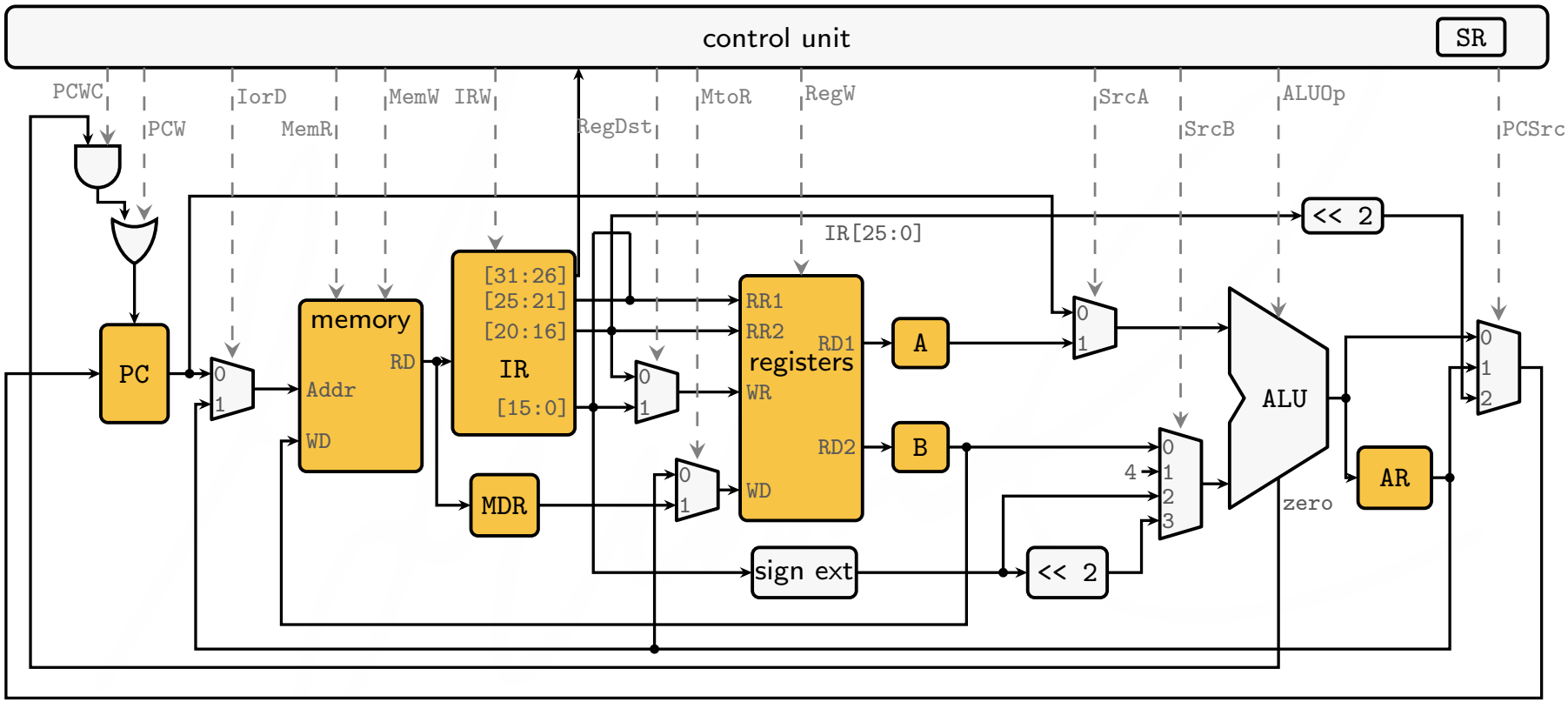
Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io



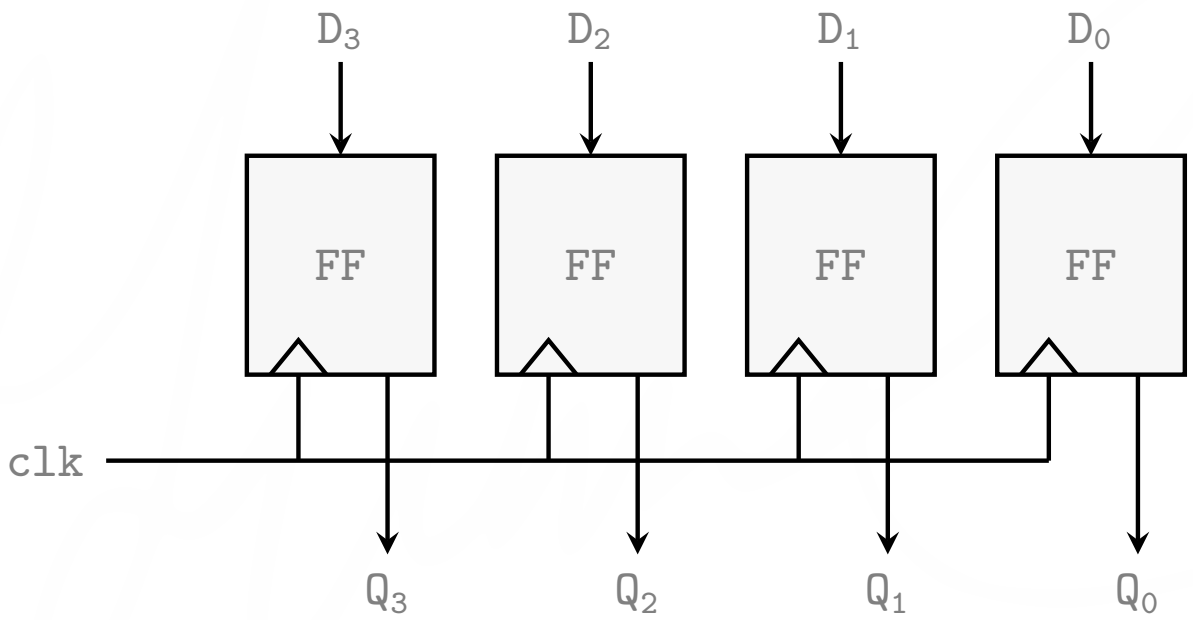
We can now build the parts of the computer that **store** values.



the PC, IR, MDR, A, B and AR are registers with special jobs, which we will meet later

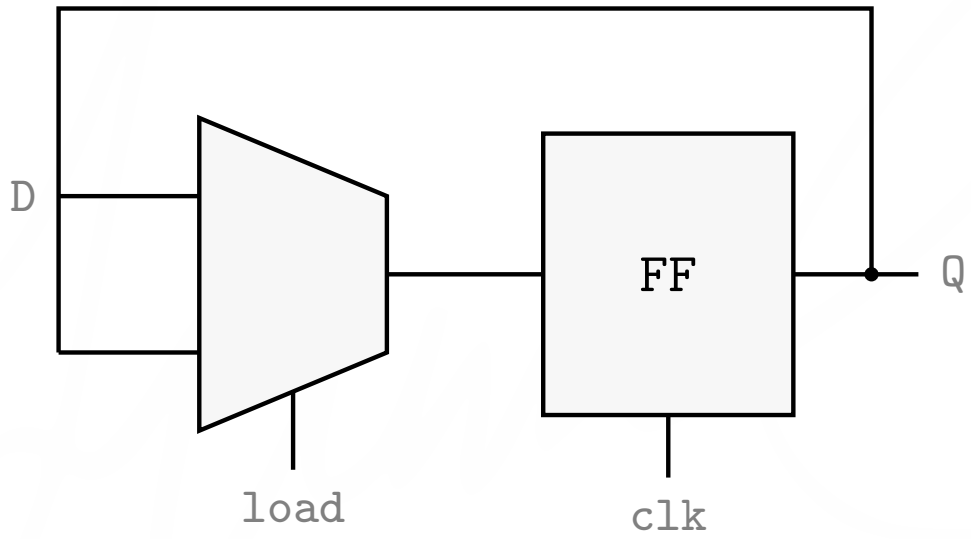
registers

Each flip-flop holds one bit. Sharing a single clock, they capture all their inputs at the same instant.

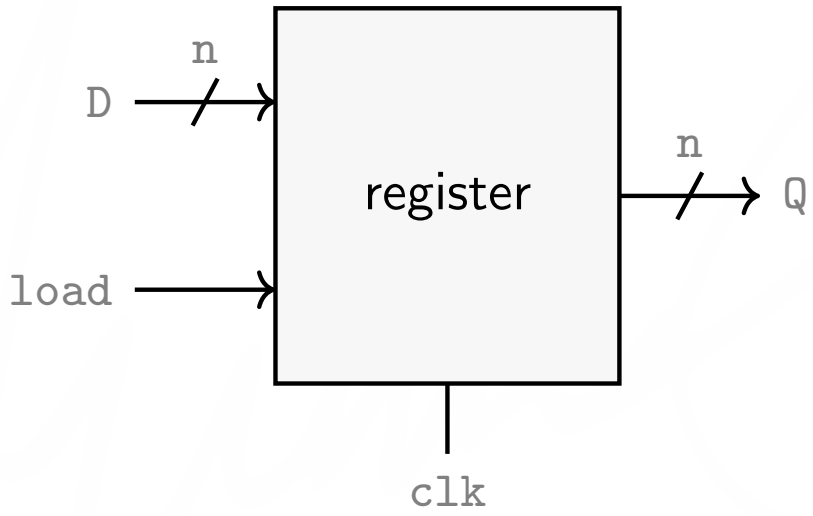


four flip-flops store a four-bit value

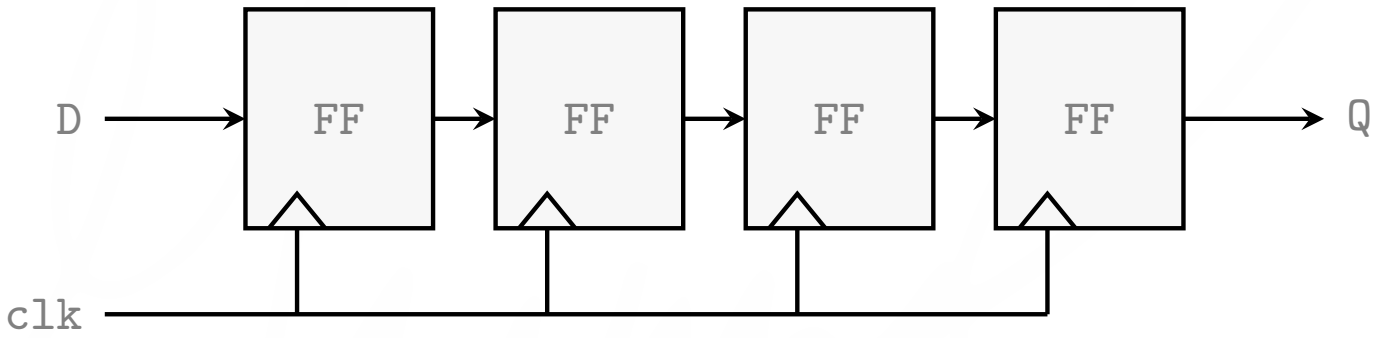
A small multiplexer chooses the next value. When `load` is 0 the flip-flop keeps its own output; when it is 1 it takes the new data.



The result is an n-bit **load register**.

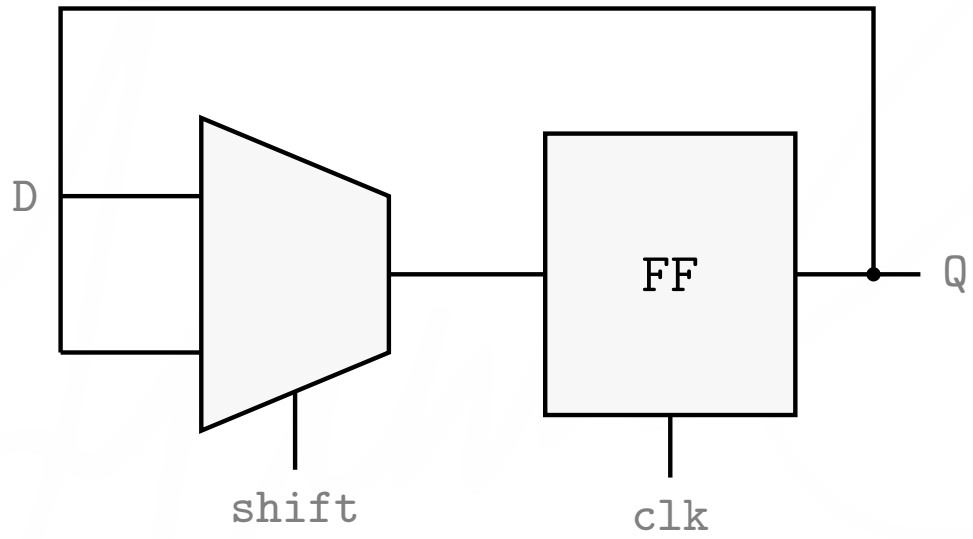


We could instead pass each flip-flop's value to its neighbour on every clock edge. The stored bits move one position at a time.

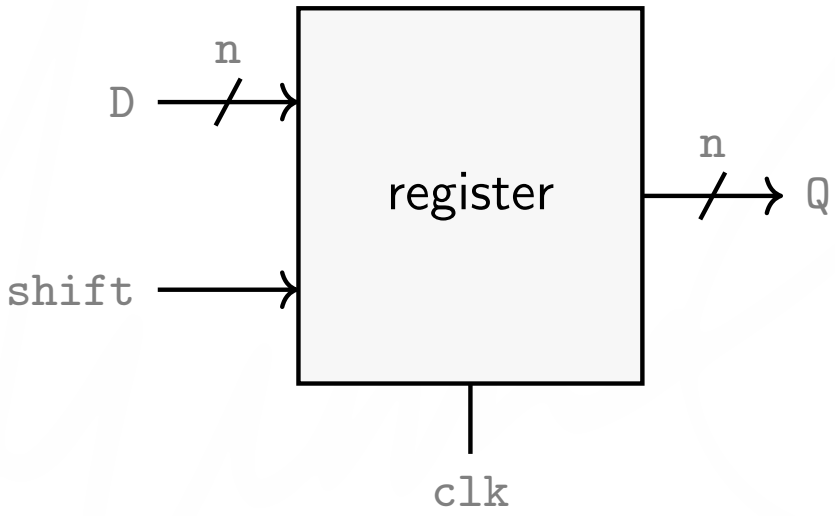


four flip-flops store a four-bit value

A small multiplexer chooses whether to shift. When `shift` is 0 the flip-flop keeps its own output; when it is 1 it shifts the existing data.

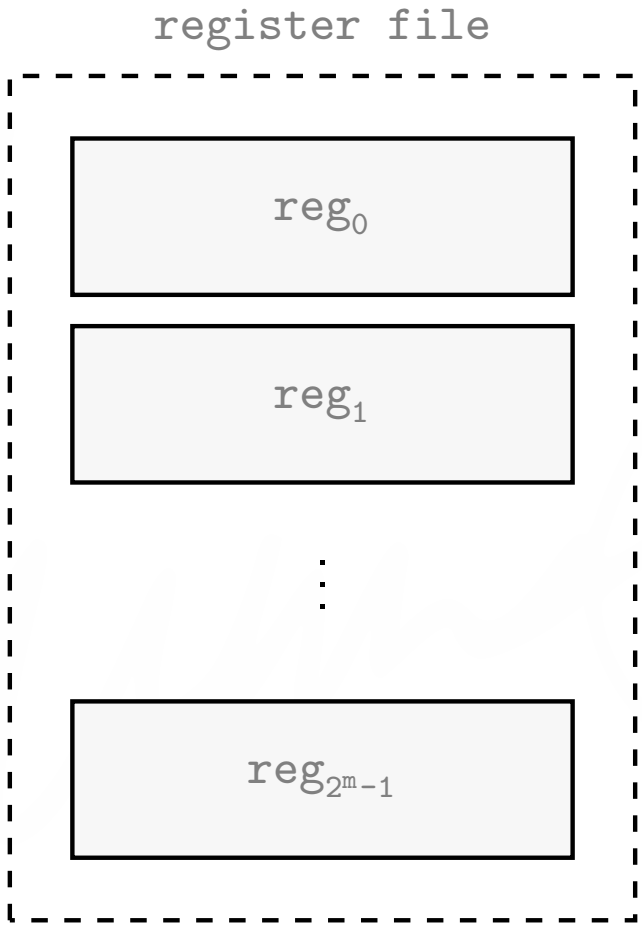


The result is an n-bit **shift register**.



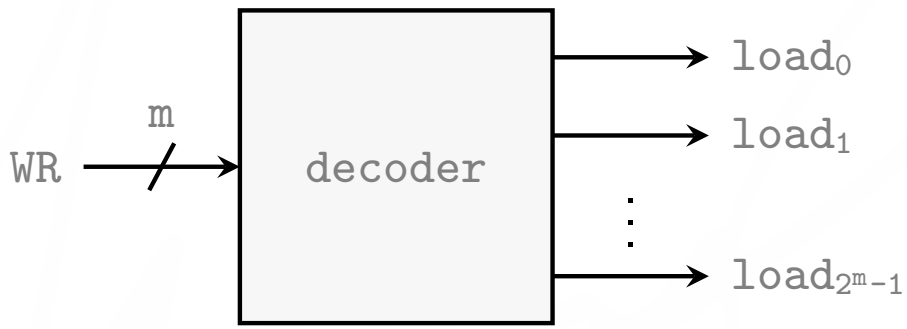
register files

A processor uses many registers, but wiring each one separately is wasteful.



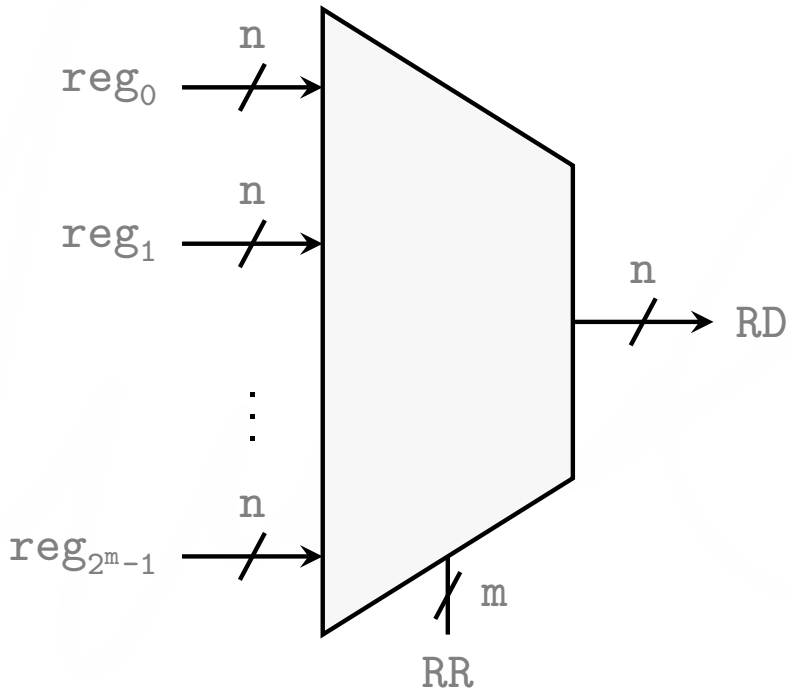
Instead, every register shares the same input and output wires.

To write a register, we activate its `load` line and no other. A **decoder** turns the `m`-bit number of the register to write, `WR`, into 2^m output lines, exactly one of which is `1`.

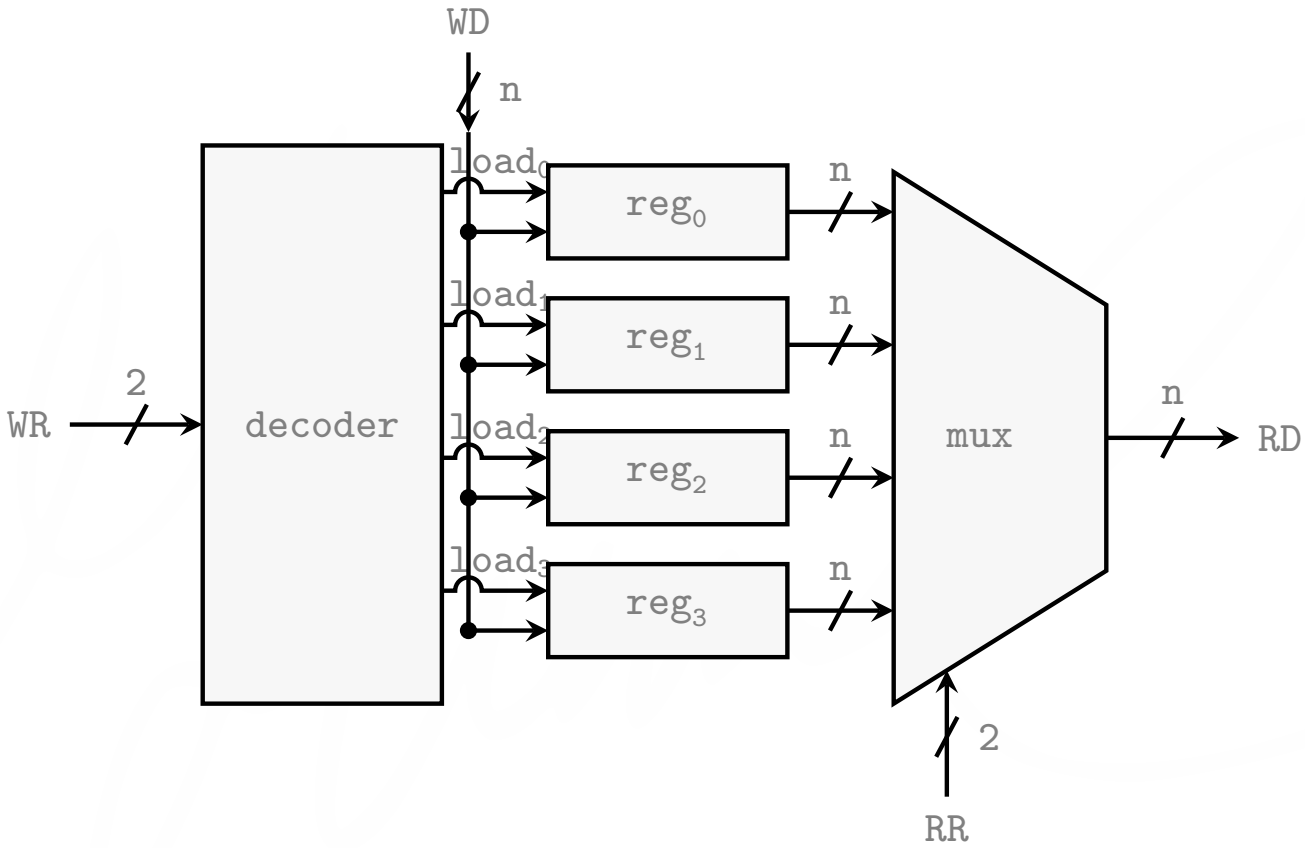


also called a one-hot decoder (one line is `1`, the rest stay `0`)

Reading goes the other direction: a **mux** uses the number of the register to read, RR, to place that register's value on the shared output wire, RD.

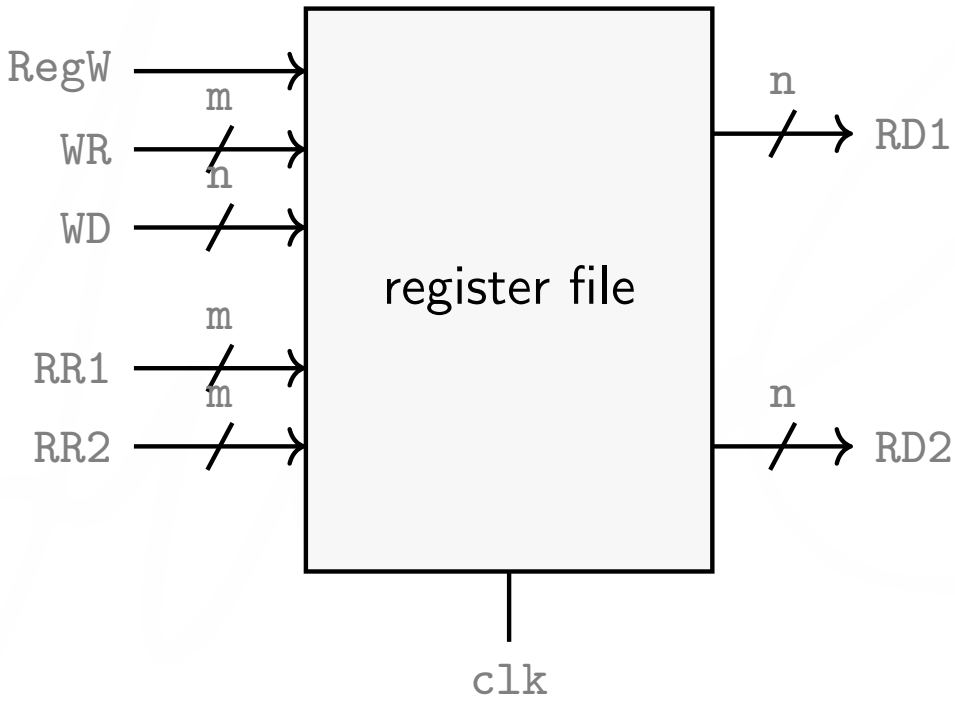


Putting it together gives us the following circuit:



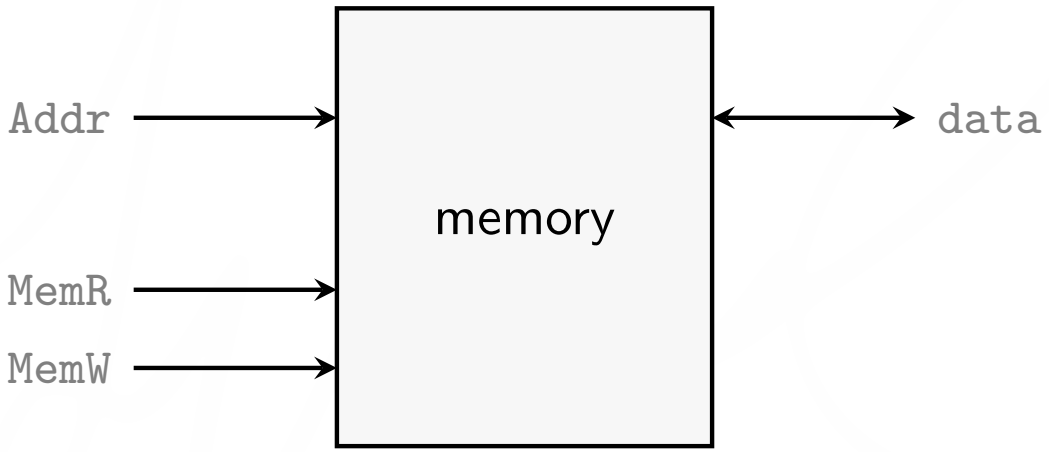
We call it a **register file**. It lives inside the processor and holds the handful of values that we need available (it's small but fast).

A real register file needs to read *two* registers and write a third, all at once — one write port and two read ports.



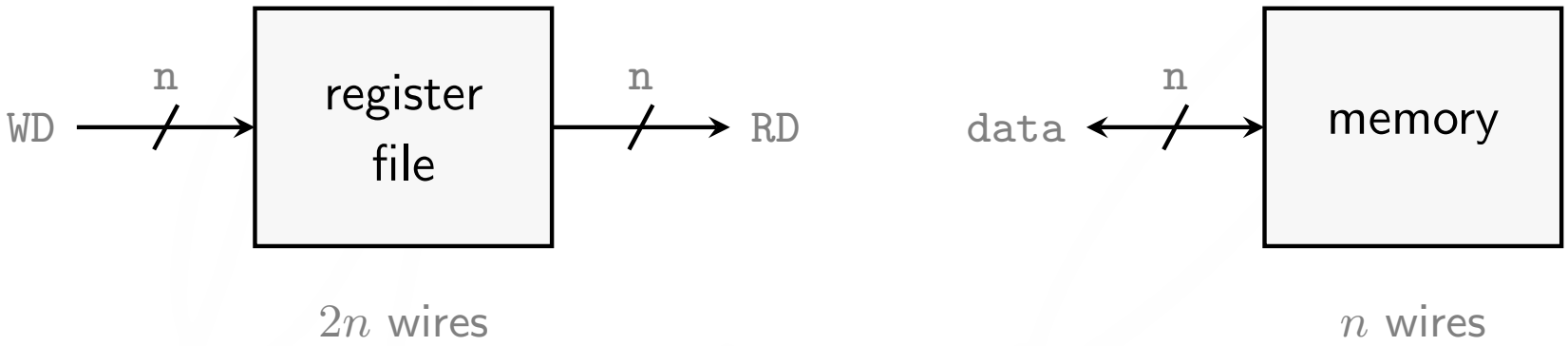
Most data will not fit in a few registers – programs and their data live in a much larger **memory** outside the processor.

It's built on the same idea as the register file. It just has vastly more locations, and is slower to access as a result.



Each memory location holds one **word**. An **address** selects the location, and **data** is read from or written to it.

Notice the one difference between the register file and memory: memory uses the same `data` line for reading and writing.

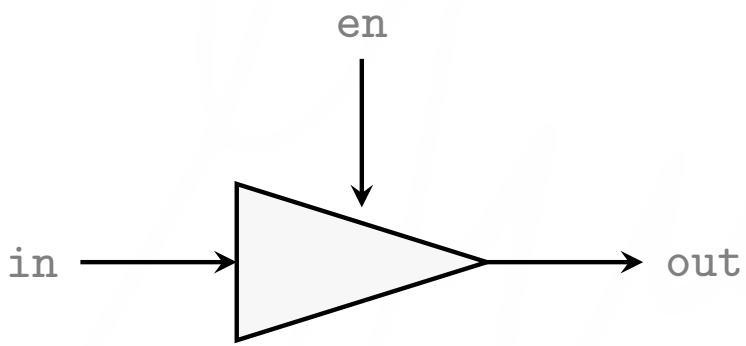


Memory sits outside the processor, so every wire to it leaves the chip through a **pin**, and pins are expensive.

And we never need both at once: in any one cycle, we either read from memory or write to it. So the same wires can take turns.

But we can't just join `WD` and `RD`. When we write, we put a new value on the wire while memory is still putting out its old one. If one is `1` and the other is `0`, the wire has no clear value.

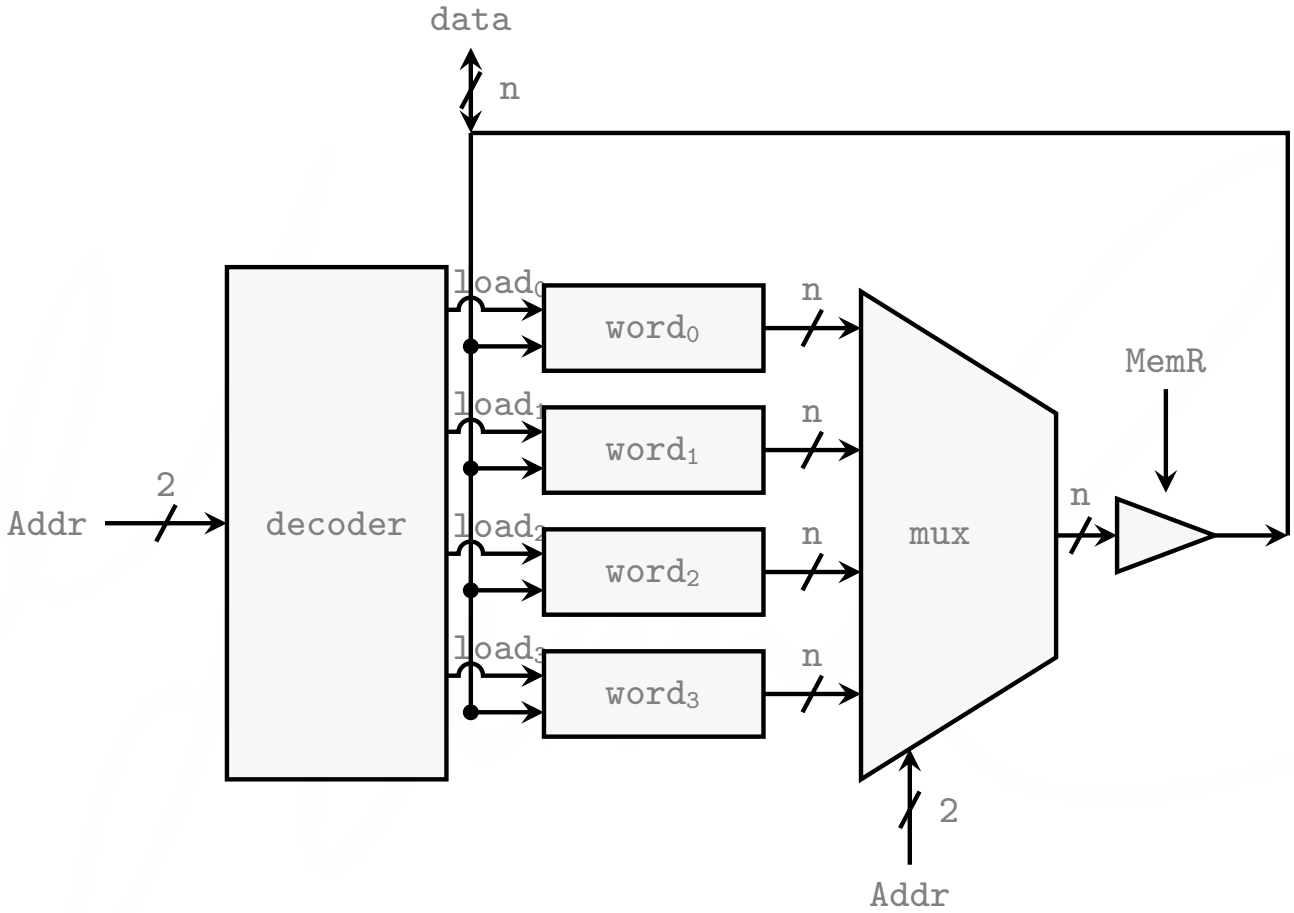
A **tri-state buffer** fixes this: its output is either driven (`0` or `1`) or disconnected, depending on its `en` line.



en	in	out
0	X	Z
1	0	0
1	1	1

`Z` means high-impedance: the buffer neither pulls the wire high nor low, so another enabled buffer elsewhere on the same wire can drive it instead

Putting it together gives us the following circuit.



This is how **memory** shares one `data` line: the tri-state buffer drives it only when we read.

Registers and memory hold values. Next we let stored state drive the circuit's behaviour, giving a finite state machine.